Hands-on Linux Systems, Networking, and Infrastructure Practice Log

Environment

- Platform: Ubuntu on UTM
- Scope: Linux networking, remote access, diagnostics, file transfer, package management, services, logging, security, containers, automation, VPN, DHCP, clustering
- Focus: command-line execution, service configuration, validation, and troubleshooting

---

1. SSH and Telnet

Objective

Understand remote access services, verify daemon processes, inspect listening ports, and compare secure vs insecure remote login.

Commands

```
sudo apt update
sudo apt install openssh-server telnetd telnet openbsd-inetd -y
sudo systemctl enable --now ssh
sudo systemctl enable --now openbsd-inetd

ps -ef | grep sshd
ps -ef | grep telnet
pgrep -a sshd
pgrep -a telnet

sudo ss -tulnp | grep -E '22|23'
sudo netstat -tulnp | grep -E '22|23'

ssh localhost
telnet localhost

who
```

```
w
last | head
```

What to observe

- `sshd` process should be visible
- SSH should listen on port 22
- Telnet should listen on port 23
- SSH session is encrypted, Telnet is plaintext
- `who` and `w` should show logged-in sessions

Outcome

Verified SSH and Telnet services, active processes, listening ports, and interactive remote sessions.

---

2. Configure and Secure SSH

Objective

Harden SSH service and validate secure configuration.

Commands

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
sudo nano /etc/ssh/sshd_config
```

Recommended settings

```
PermitRootLogin no
PasswordAuthentication yes
PubkeyAuthentication yes
MaxAuthTries 3
PermitEmptyPasswords no
X11Forwarding no
Protocol 2
```

Validate and restart

```
sudo sshd -t
sudo systemctl restart ssh
sudo systemctl status ssh
```

What to observe

- `sshd -t` should return no syntax errors
- SSH service should restart successfully
- Root login should be disabled if configured

Outcome

Configured a more secure SSH service with controlled authentication behavior.

---

3. SSH Keys - Access Remote Server Without Password

Objective

Generate SSH keys and enable passwordless login.

Commands

```
ssh-keygen -t ed25519 -C "lab-key"
ls -la ~/.ssh
cat ~/.ssh/id_ed25519.pub
ssh-copy-id $USER@localhost
ssh localhost
```

Manual method

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Optional hardening after validation

sudo nano /etc/ssh/sshd_config

Set:

PasswordAuthentication no
PubkeyAuthentication yes
PermitRootLogin no

Then:

sudo sshd -t
sudo systemctl restart ssh

What to observe

- Login should work without password after key installation
- `authorized_keys` should contain the public key

Outcome

Implemented passwordless SSH access using public key authentication.

---

4. The `ss` Command

Objective

Inspect listening ports, established connections, and socket details.

Commands

ss -tuln
ss -tulpn
ss -tn
ss -tnp
ss -s
ss -ltn
ss -ludp

What to observe

- Open TCP and UDP ports
- Associated services and PIDs
- Established remote sessions

Outcome

Used `ss` for network socket inspection and service-level visibility.

---

5. curl and ping Commands

Objective

Test connectivity, web access, and basic application-layer reachability.

Commands

```
ping -c 4 8.8.8.8
ping -c 4 google.com
curl http://example.com
curl -I http://example.com
curl ifconfig.me
curl -v http://example.com
```

What to observe

- Successful ICMP replies
- DNS resolution behavior
- HTTP headers and response codes

Outcome

Validated network connectivity and HTTP reachability using diagnostic commands.

---

6. Build a Simple Network Troubleshooting Toolkit

Objective

Combine key commands into a reusable troubleshooting workflow.

Commands

```
ip a
ip route
ping -c 4 8.8.8.8
ping -c 4 google.com
ss -tulnp
curl -I http://example.com
nslookup google.com
dig google.com
traceroute google.com
```

What to observe

- Interface status
- Default route
- IP connectivity
- DNS resolution
- Port listening
- Path to destination

Outcome

Built a basic workflow for diagnosing common network issues.

---

7. FTP - File Transfer Protocol

Objective

Install and test FTP service.

Commands

```
sudo apt install vsftpd ftp -y
sudo systemctl enable --now vsftpd
sudo systemctl status vsftpd
sudo ss -tulnp | grep :21
ftp localhost
```

What to observe

- FTP service on port 21
- Successful anonymous or local login depending on configuration

Outcome

Configured and tested FTP-based file transfer service.

---

8. SCP - Secure Copy Protocol

Objective

Transfer files securely over SSH.

Commands

```
echo "test file" > file1.txt
scp file1.txt $USER@localhost:/tmp/
ls /tmp/file1.txt
```

Copy from remote

```
scp $USER@localhost:/tmp/file1.txt .
```

What to observe

- File transfer should succeed over SSH

- Authentication should follow SSH rules

Outcome

Transferred files securely using SCP.

---

9. rsync - Remote Synchronization

Objective

Synchronize files and directories efficiently.

Commands

```
mkdir -p ~/srcdir ~/destdir
echo "sample" > ~/srcdir/a.txt
rsync -av ~/srcdir/ ~/destdir/
ls ~/destdir

echo "new line" >> ~/srcdir/a.txt
rsync -av ~/srcdir/ ~/destdir/
```

Remote example

```
rsync -av ~/srcdir/ $USER@localhost:/tmp/rsync-test/
```

What to observe

- Only changed files should be synchronized
- Permissions and timestamps can be preserved

Outcome

Used `rsync` for efficient local and remote file synchronization.

---

10. Downloading Files or Apps with wget

Objective

Download files from remote sources using command line.

Commands

wget http://example.com
wget -O index.html http://example.com
wget -c http://example.com/file.zip
wget --spider http://example.com

What to observe

- Download progress
- Resume behavior with `-c`
- URL availability with `--spider`

Outcome

Downloaded and validated files using `wget`.

---

11. Create a File Transfer and Synchronization Toolkit

Objective

Use multiple transfer tools depending on security and use case.

Commands

ftp localhost
scp file1.txt $USER@localhost:/tmp/
rsync -av ~/srcdir/ ~/destdir/

wget http://example.com

What to observe

- FTP for legacy/plain transfer
- SCP for secure copy
- rsync for synchronization
- wget for downloads

Outcome

Practiced choosing tools based on purpose and security requirements.

---

12. System Updates and Repositories (rpm, yum / apt equivalent)

Objective

Understand package management and patch workflows.

Ubuntu commands

sudo apt update
sudo apt list --upgradable
sudo apt upgrade -y
sudo apt install nginx -y
sudo apt remove nginx -y
sudo apt autoremove -y

RHEL-style reference

sudo yum check-update
sudo yum update -y
sudo yum install nginx -y
sudo rpm -qa | head
sudo rpm -qi bash

Outcome

Managed packages, updates, and software lifecycle operations.

---

13. System Upgrade and Patch Management

Objective

Perform package updates and review patch state.

Commands

```
sudo apt update
sudo apt upgrade -y
uname -r
apt list --installed | head
```

What to observe

- Updated packages
- Kernel or package version changes

Outcome

Applied system patches and reviewed package state after upgrade.

---

14. Create Local Repository

Ubuntu-style basic local repo idea

```
mkdir -p ~/localrepo
cp *.deb ~/localrepo/
cd ~/localrepo
```

dpkg-scanpackages . /dev/null | gzip -9c > Packages.gz

RHEL reference

createrepo /var/www/html/localrepo

Outcome

Understood local repository structure and package indexing.

---

15. Advanced Package Management

Commands

apt-cache search nginx
apt show nginx
dpkg -l | grep ssh
dpkg -L openssh-server
apt depends nginx
apt rdepends openssh-client

Outcome

Explored package metadata, dependencies, and installed files.

---

16. Rollback Patches and Updates

Commands

sudo apt remove package-name
sudo apt install package-name=version
apt-cache policy package-name

RHEL reference

yum history

yum history undo <id>

Outcome

Reviewed package versioning and rollback concepts.

---

17. Build a Network Diagnostics and Configuration Console

Objective

Combine diagnostics into a reusable shell menu.

Simple script

nano net-console.sh

Paste:

```
#!/bin/bash
echo "1. Show IP"
echo "2. Show Routes"
echo "3. Show Ports"
echo "4. Ping Google"
read -p "Choose: " ch
case $ch in
 1) ip a ;;
 2) ip route ;;
 3) ss -tulnp ;;
 4) ping -c 4 google.com ;;
 *) echo "Invalid" ;;
esac
```

Run:

```
chmod +x net-console.sh
./net-console.sh
```

Outcome

Created a simple command-line admin utility.

---

18. DNS - Install and Configure

Commands

```
sudo apt install bind9 dnsutils -y
sudo systemctl enable --now bind9
sudo systemctl status bind9
ss -tulnp | grep :53
```

Verification

```
nslookup google.com
dig google.com
```

Outcome

Installed DNS service and tested resolution tools.

---

19. nslookup and dig

Commands

```
nslookup google.com
nslookup openai.com
dig google.com
dig openai.com
dig google.com MX
dig google.com NS
```

What to observe

- A, MX, and NS records
- Resolver response structure

Outcome

Queried DNS records using command-line tools.

---

20. NTP, chronyd, and timedatectl

Commands

```
timedatectl
sudo apt install chrony -y
sudo systemctl enable --now chrony
sudo systemctl status chrony
chronyc tracking
chronyc sources
timedatectl set-timezone America/New_York
```

Outcome

Configured and validated time synchronization and timezone management.

---

21. Mail Transfer Agent

Commands

```
sudo apt install postfix mailutils -y
sudo systemctl enable --now postfix
sudo systemctl status postfix
echo "Test mail body" | mail -s "Test Subject" $USER
mail
```

Outcome

Installed a local MTA and tested simple mail delivery.

---

22. Web Server - Apache

Commands

```
sudo apt install apache2 -y
sudo systemctl enable --now apache2
sudo systemctl status apache2
ss -tulnp | grep :80
curl http://localhost
echo "<h1>Apache Test</h1>" | sudo tee /var/www/html/index.html
curl http://localhost
```

Outcome

Installed and managed Apache HTTP server and served a test page.

---

23. Nginx

Commands

```
sudo apt install nginx -y
sudo systemctl enable --now nginx
sudo systemctl status nginx
ss -tulnp | grep :80
curl http://localhost
```

Outcome

Installed and managed Nginx service and validated web response.

---

24. Cockpit

Commands

```
sudo apt install cockpit -y
sudo systemctl enable --now cockpit.socket
sudo systemctl status cockpit.socket
ss -tulnp | grep :9090
```

Open:

https://localhost:9090

Outcome

Enabled Linux web-based administration using Cockpit.

---

25. Proxy Server in Linux (Squid)

Commands

```
sudo apt install squid -y
sudo systemctl enable --now squid
sudo systemctl status squid
ss -tulnp | grep :3128
grep -v '^#' /etc/squid/squid.conf | head
```

Outcome

Installed proxy service and reviewed default configuration behavior.

---

26. Central Logger (rsyslog)

Commands

```
sudo systemctl enable --now rsyslog
sudo systemctl status rsyslog
logger "This is a test log entry"
sudo tail -f /var/log/syslog
```

Outcome

Generated and observed centralized system logs using rsyslog.

---

27. Nagios

Commands

Install package if available in repo, then:

```
sudo systemctl status nagios
ss -tulnp | grep nagios
```

Outcome

Explored monitoring platform installation and service presence.

---

28. Securing Linux Machine (OS Hardening)

Commands

```
sudo ufw enable
sudo ufw status verbose
sudo passwd -l root
sudo systemctl list-unit-files --type=service | grep enabled
sudo apt install fail2ban -y
sudo systemctl enable --now fail2ban
sudo systemctl status fail2ban
```

Outcome

Applied baseline security controls including firewalling and service hardening.

---

29. OpenLDAP

Commands

```
sudo apt install slapd ldap-utils -y
sudo systemctl status slapd
ldapsearch -x -LLL -H ldap:/// -b dc=example,dc=com
```

Outcome

Installed LDAP components and tested basic directory query.

---

30. Tracing Network Traffic (traceroute)

Commands

```
sudo apt install traceroute -y
traceroute google.com
tracepath google.com
```

Outcome

Observed hop-by-hop path to a remote destination.

---

31. Open Image File Through Command Line

Commands

xdg-open image.png
file image.png
ls -lh image.png

Outcome

Opened and inspected image files from the command line.

---

32. Firewall

Commands

sudo ufw status
sudo ufw enable
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw deny 23/tcp
sudo ufw status numbered

Outcome

Configured firewall rules to allow secure services and block insecure ones.

---

33. Tune System Performance (tuned, nice, renice)

Commands

nice -n 10 dd if=/dev/zero of=/tmp/testfile bs=1M count=200
ps -eo pid,ni,cmd | head
sudo renice 5 -p <PID>
top

Outcome

Observed process priority management and basic performance tuning controls.

---

34. Develop an Admin Console for Logs, Traffic, and Processes

Script example

nano admin-console.sh

Paste:

```
#!/bin/bash
echo "1. Logs"
echo "2. Traffic"
echo "3. Processes"
read -p "Choice: " ch
case $ch in
 1) tail -n 20 /var/log/syslog ;;
 2) ss -tulnp ;;
 3) ps -ef | head -20 ;;
 *) echo "Invalid choice" ;;
esac
```

Run:

```
chmod +x admin-console.sh
./admin-console.sh
```

Outcome

Built a small operational dashboard from shell commands.

---

35. Run Containers - Podman

Commands

```
sudo apt install podman -y
podman --version
podman pull docker.io/library/nginx
podman run -d --name mynginx -p 8080:80 nginx
podman ps
curl http://localhost:8080
```

Outcome

Deployed a containerized service using Podman.

---

36. Docker

Commands

```
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo docker version
sudo docker pull nginx
sudo docker run -d --name nginx1 -p 8081:80 nginx
sudo docker ps
curl http://localhost:8081
```

Outcome

Installed Docker and ran a containerized web service.

---

37. Kickstart / Automated Linux Installation

Objective

Review unattended installation concepts.

Practical step

- Study kickstart or autoinstall file structure
- Define hostname, user, packages, partitioning, post-install scripts

Outcome

Understood automation of Linux OS deployment.

---

38. Ansible

Commands

```
sudo apt install ansible -y
ansible --version
echo "localhost ansible_connection=local" > inventory
ansible -i inventory all -m ping
ansible -i inventory all -m shell -a "uptime"
```

Outcome

Validated Ansible installation and executed basic automation tasks.

---

39. OpenVPN

Commands

```
sudo apt install openvpn -y
openvpn --version
systemctl status openvpn
```

Outcome

Installed VPN tooling and reviewed service availability.

---

40. DHCP Server

Commands

```
sudo apt install isc-dhcp-server -y
sudo systemctl status isc-dhcp-server
```

Edit config:

```
sudo nano /etc/dhcp/dhcpd.conf
```

Example:

```
subnet 192.168.65.0 netmask 255.255.255.0 {
 range 192.168.65.100 192.168.65.150;
 option routers 192.168.65.1;
 option domain-name-servers 8.8.8.8;
}
```

Outcome

Configured basic DHCP scope structure and reviewed service behavior.

---

41. Clustered Environment on Linux

Practical exposure

- Study node communication
- identify shared services
- review heartbeat/failover concepts
- document active/passive roles

Commands

```
hostnamectl
ip a
ss -tulnp
```

Outcome

Explored clustered environment concepts and supporting service visibility.

---

42. Role Play / Operational Scenarios

Scenario 1: Diagnosing Network Issues and Securing SSH Access

Commands used:

```
ip a
ip route
ping -c 4 8.8.8.8
ping -c 4 google.com
ss -tulnp
ps -ef | grep sshd
sudo systemctl status ssh
sudo sshd -t
```

Scenario 2: Choosing the Right File Transfer Tool

Commands used:

```
ftp localhost
scp file1.txt $USER@localhost:/tmp/
rsync -av ~/srcdir/ ~/destdir/
wget http://example.com
```

Scenario 3: Managing System Updates and Automating Server Configuration

Commands used:

```
sudo apt update
sudo apt upgrade -y
ansible -i inventory all -m ping
```

Scenario 4: Setting Up a Web Server and Enabling Web-Based Administration

Commands used:

```
sudo systemctl status apache2
sudo systemctl status nginx
sudo systemctl status cockpit.socket
ss -tulnp | grep -E '80|9090'
```

Scenario 5: Monitoring System Performance and Using Containers Efficiently

Commands used:

```
top
ps -ef
nice
renice
podman ps
docker ps
```

Key Learning Summary

This hands-on practice reinforced several important areas:

- remote access and secure service administration
- network diagnostics and troubleshooting
- file transfer and synchronization methods
- package and update management
- DNS, time, web, proxy, logging, and mail services
- OS hardening and firewall control
- containers and automation
- operational troubleshooting using real Linux commands

Short Portfolio Summary

Worked through practical Linux labs covering SSH hardening, key-based authentication, socket inspection with `ss`, diagnostics using `curl`, `ping`, `dig`, and `traceroute`, file transfer using FTP/SCP/rsync/wget, package management, DNS and time services, Apache/Nginx/Cockpit/Squid/rsyslog, firewalling, OS hardening, containers with Podman and Docker, Ansible automation, OpenVPN, DHCP, and clustered environment fundamentals.